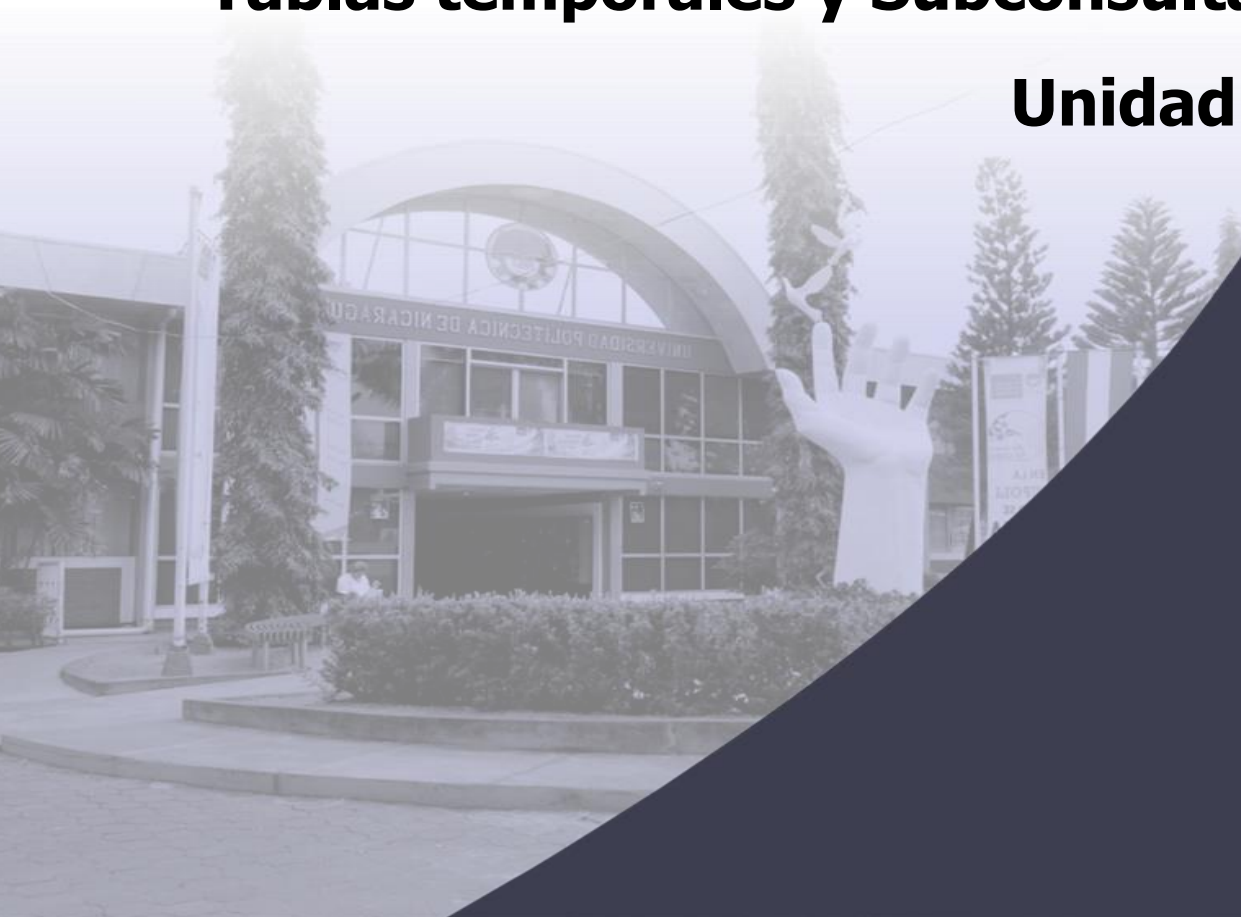


Unidad Didáctica

**Consultas avanzadas: Uniones. Joins,
Tablas temporales y Subconsultas,
Unidad II**



Contenido

Introducción	3
Desarrollo de Contenidos	5
Consultas de 2 a más tablas por medio de Where	5
JOIN	8
INNER JOIN	8
RIGHT JOIN	10
LEFT JOIN.....	12
FULL JOIN.....	14
UNION	16
Tablas Temporales.....	17
SubConsultas.....	21
Subconsultas en SELECT	21
Subconsultas en FROM.....	23
Subconsultas en WHERE	25
Bibliografía y Webgrafía	28
Glosario	28

Introducción

En el amplio mundo de las bases de datos y la gestión de información, existen fascinantes herramientas y técnicas que permiten obtener conocimientos profundos y valiosos a partir de datos dispersos. Entre ellas, destacan las consultas de 2 o más tablas mediante la cláusula "WHERE", una poderosa habilidad que nos permite explorar la interrelación de datos en un nivel más complejo y enriquecedor. Adentrarse en este tema nos permitirá descubrir cómo combinar información de diferentes tablas y obtener resultados más significativos, desafiándonos intelectualmente y abriendo nuevas perspectivas para el análisis de datos.

Otro concepto imprescindible en el mundo de las bases de datos es el uso de "JOIN", una operación que nos permite combinar filas de dos o más tablas en función de una relación establecida entre ellas. Al dominar esta técnica, se nos revelará un universo de posibilidades para la generación de datos consolidados y la obtención de información más completa y contextualizada. ¡Es como un puzzle intelectual que, una vez resuelto, revela un cuadro impresionante de conocimiento!

En el entorno académico, el concepto de "UNION" cobra una importancia especial. Esta operación nos brinda la habilidad de combinar resultados de consultas independientes y crear una vista integral de la información. Cada tabla, como un lienzo en blanco, aporta piezas valiosas al rompecabezas de nuestro análisis, y la unión de todas ellas nos otorga una imagen más clara y precisa de los datos. Así, podremos afrontar retos más ambiciosos en nuestras investigaciones y planteamientos, llevando nuestras habilidades analíticas al siguiente nivel.

Por otro lado, el uso de tablas temporales es una estrategia inteligente y eficiente para manejar datos que cambian constantemente o para realizar cálculos intermedios en el proceso de análisis. Estas tablas, como burbujas temporales, nos permiten encapsular datos relevantes para ciertas operaciones sin afectar el conjunto de la base de datos. Su implementación nos brinda la flexibilidad necesaria

para realizar experimentos, comparaciones y comprobaciones de hipótesis sin poner en riesgo la integridad de la información.

Finalmente, las subconsultas se presentan como una joya oculta en el vasto universo de la gestión de datos. Estas consultas dentro de consultas nos abren las puertas a un mundo de posibilidades, permitiéndonos acceder a datos más específicos y realizar operaciones complejas en pasos discretos. Como arqueólogos de la información, nos aventuramos en capas más profundas y misteriosas de la base de datos, descubriendo tesoros de conocimiento que de otra manera permanecerían ocultos a simple vista.

En este apasionante recorrido por los entresijos de las consultas de múltiples tablas, las operaciones JOIN y UNION, el uso de tablas temporales y las subconsultas, esperamos despertar en cada estudiante y académico el deseo ardiente de explorar y comprender estos conceptos clave. Cada uno de ellos representa una ventana hacia la comprensión más profunda de los datos y abre un mundo de posibilidades para el avance en el campo de la gestión y análisis de la información. Así, nos preparamos para afrontar con confianza los retos académicos y profesionales que nos aguardan en este emocionante viaje de descubrimiento y aprendizaje. ¡Adelante, exploradores del conocimiento!

Consultas de 2 a más tablas por medio de Where

Cuando realizas consultas con dos tablas utilizando la cláusula WHERE sin JOIN, estás efectivamente aplicando condiciones de filtrado en las columnas de ambas tablas y cruzando los resultados en función de las condiciones establecidas.

Por ejemplo, consideremos esta consulta:

```
SELECT V.*, P.nombre AS nombre_producto  
  
FROM Ventas V, Vendedores VD, Productos P  
  
WHERE V.cod_vendedor = VD.id_vendedor  
  
AND V.cod_producto = P.id_producto  
  
AND VD.nombre = 'NombreVendedor';
```

Aquí, estamos obteniendo información de las tablas Ventas, Vendedores y Productos. En la cláusula WHERE, establecemos condiciones para relacionar las tablas y filtrar los resultados. Estas condiciones de relación se especifican mediante las comparaciones entre columnas de las tablas involucradas (V.cod_vendedor = VD.id_vendedor y V.cod_producto = P.id_producto).

Diferencia con el uso de JOIN:

- La cláusula JOIN es una forma más estructurada y legible de combinar información de varias tablas. Permite especificar explícitamente cómo se relacionan las tablas en lugar de depender únicamente de condiciones de filtro en la cláusula WHERE.

- Al usar JOIN, estás indicando la relación entre las tablas de manera más clara y separando la lógica de combinación de la lógica de filtrado. Esto facilita la comprensión del propósito de la consulta y permite a los optimizadores de bases de datos realizar mejoras de rendimiento al ejecutar la consulta.

En resumen, aunque es posible realizar consultas utilizando WHERE sin JOIN, el uso de la cláusula JOIN es más recomendable en términos de claridad, mantenibilidad y optimización del rendimiento de las consultas. Proporciona una estructura más explícita para especificar cómo se relacionan las tablas y separa la lógica de combinación de la lógica de filtro.

Ejemplos

1. Obtener todas las ventas de un vendedor específico sin usar JOIN:

```
SELECT V.*, P.nombre AS nombre_producto  
  
FROM Ventas V, Vendedores VD, Productos P  
  
WHERE V.cod_vendedor = VD.id_vendedor  
  
AND V.cod_producto = P.id_producto  
  
AND VD.nombre = 'NombreVendedor';
```

En este ejemplo, estamos usando una lista separada por comas de las tablas involucradas en el FROM. Luego, en la cláusula WHERE, especificamos las condiciones para relacionar las tablas y filtrar los resultados.

2. Encontrar los productos vendidos en una fecha particular sin usar JOIN:

```
SELECT P.nombre AS nombre_producto, V.fecha, V.kilos  
  
FROM Ventas V, Productos P  
  
WHERE V.cod_producto = P.id_producto  
  
AND V.fecha = '2023-08-27';
```

En esta consulta, estamos relacionando las tablas Ventas y Productos en la cláusula FROM y utilizando la cláusula WHERE para aplicar las condiciones de filtro.

3. Recuperar los nombres de los grupos y la cantidad de productos en cada grupo sin usar JOIN:

```
SELECT G.nombre_grupo, COUNT(P.id_producto) AS cantidad_productos  
  
FROM Grupos G, Productos P  
  
WHERE G.id_grupo = P.id_grupo  
  
GROUP BY G.nombre_grupo;
```

Aquí, estamos combinando las tablas Grupos y Productos en la cláusula FROM y utilizando la cláusula WHERE para establecer la condición de relación. Usamos la cláusula GROUP BY para contar la cantidad de productos en cada grupo.

Es importante destacar que el uso de la sintaxis sin JOIN puede ser menos claro y puede resultar en consultas más difíciles de mantener y entender. Por lo general, se recomienda el uso de la cláusula JOIN para realizar consultas más eficientes y legibles.

JOIN

INNER JOIN

El INNER JOIN es uno de los tipos más comunes de operaciones de unión en SQL.

Permite combinar registros de dos o más tablas basándose en una condición específica y solo devuelve las filas donde existe una coincidencia entre las tablas involucradas en la condición de unión.

Funcionamiento del INNER JOIN: Cuando usas INNER JOIN en una consulta, le estás indicando a la base de datos que combine registros de dos tablas solo cuando se cumpla una condición de unión especificada. El resultado es una tabla resultante que contiene únicamente las filas que tienen coincidencias en ambas tablas, según la condición establecida.

La sintaxis general del INNER JOIN es la siguiente:

```
SELECT columnas
```

```
FROM tabla1
```

```
INNER JOIN tabla2
```

```
ON tabla1.columna = tabla2.columna;
```

La condición de unión se establece mediante la cláusula ON, donde defines cómo se relacionan las tablas. La consulta examina cada fila de tabla1 y busca coincidencias en tabla2 basándose en la condición de unión. Si hay una coincidencia, se incluye en el resultado.

Casos de uso del INNER JOIN:

El INNER JOIN es adecuado cuando deseas obtener registros que tengan una relación en ambas tablas. Aquí hay dos ejemplos de casos en los que podrías usar INNER JOIN:

1- Obtener ventas con información del vendedor y producto:

Supongamos que deseas obtener un informe de ventas que incluya detalles del vendedor y el producto para cada venta.

```
SELECT V.*, VD.nombre AS nombre_vendedor, P.nombre AS nombre_producto  
  
FROM Ventas V  
  
INNER JOIN Vendedores VD ON V.cod_vendedor = VD.id_vendedor  
  
INNER JOIN Productos P ON V.cod_producto = P.id_producto;
```

En este ejemplo, utilizamos dos INNER JOIN para combinar las tablas Ventas, Vendedores y Productos. Esto nos permite obtener información detallada sobre cada venta, incluyendo el nombre del vendedor y el nombre del producto vendido.

2. Encontrar vendedores que han realizado ventas en un grupo específico:

Supongamos que deseas encontrar los vendedores que han realizado ventas de productos pertenecientes a un grupo específico.

```
SELECT VD.nombre, G.nombre_grupo  
  
FROM Vendedores VD  
  
INNER JOIN Ventas V ON VD.id_vendedor = V.cod_vendedor
```

```
INNER JOIN Productos P ON V.cod_producto = P.id_producto
```

```
INNER JOIN Grupos G ON P.id_grupo = G.id_grupo
```

```
WHERE G.nombre_grupo = 'Frutas';
```

En este caso, utilizamos múltiples INNER JOIN para relacionar las tablas Vendedores, Ventas, Productos y Grupos. El objetivo es encontrar los nombres de los vendedores que han realizado ventas de productos en el grupo de "Frutas".

En resumen, el INNER JOIN es útil cuando necesitas combinar información de diferentes tablas basándote en relaciones específicas. Se utiliza para obtener datos que tienen correspondencias en ambas tablas y es fundamental para consultas que involucran múltiples fuentes de información relacionadas.

RIGHT JOIN

El RIGHT JOIN es otro tipo de operación de unión en SQL que permite combinar registros de dos o más tablas, pero con un enfoque diferente al INNER JOIN. Mientras que el INNER JOIN devuelve solo las filas que tienen coincidencias en ambas tablas, el RIGHT JOIN **devuelve todas las filas de la tabla de la derecha (la segunda tabla en la cláusula JOIN) y las filas coincidentes de la tabla de la izquierda (la primera tabla en la cláusula JOIN), completando con valores nulos donde no hay coincidencias.**

Funcionamiento del RIGHT JOIN:

Cuando usas RIGHT JOIN en una consulta, estás solicitando que se devuelvan todas las filas de la tabla de la derecha, junto con las filas coincidentes de la tabla de la

izquierda. Si no hay coincidencias en la tabla de la izquierda, se llenarán los campos con valores nulos.

La sintaxis general del RIGHT JOIN es la siguiente:

```
SELECT columnas
```

```
FROM tabla1
```

```
RIGHT JOIN tabla2
```

```
ON tabla1.columna = tabla2.columna;
```

La consulta examina cada fila de tabla2 y busca coincidencias en tabla1 basándose en la condición de unión. Si hay una coincidencia, se incluye en el resultado junto con los datos de tabla2. Si no hay coincidencia en tabla1, los campos relacionados de esa tabla tendrán valores nulos en el resultado.

Ejemplo de uso del RIGHT JOIN:

Supongamos que tenemos una base de datos de empleados y departamentos, y queremos obtener una lista de todos los departamentos y los empleados asociados a cada uno, incluso si algunos departamentos no tienen empleados.

```
SELECT D.nombre AS departamento, E.nombre AS empleado
```

```
FROM Departamentos D
```

```
RIGHT JOIN Empleados E ON D.id_departamento = E.id_departamento;
```

En este ejemplo, utilizamos un RIGHT JOIN para combinar las tablas Departamentos y Empleados. La consulta devuelve todos los departamentos junto con los nombres de los empleados asociados a cada uno. Si un departamento no tiene empleados,

su nombre aparecerá en la columna "departamento" y la columna "empleado" tendrá un valor nulo.

Nota: Es importante mencionar que, en la mayoría de las bases de datos, el RIGHT JOIN es menos común que el INNER JOIN y el LEFT JOIN. Esto se debe a que muchas veces la tabla de la izquierda (primera tabla en la cláusula JOIN) es la tabla principal que contiene la información principal que se desea mostrar en el resultado. Sin embargo, en algunas situaciones específicas, el RIGHT JOIN puede ser útil para obtener todos los registros de la tabla de la derecha, incluso si no hay coincidencias en la tabla de la izquierda.

LEFT JOIN

El LEFT JOIN es otro tipo de operación de unión en SQL que permite combinar registros de dos o más tablas. Al igual que el RIGHT JOIN, el LEFT JOIN también tiene su enfoque único. Mientras que el INNER JOIN devuelve solo las filas que tienen coincidencias en ambas tablas, el LEFT JOIN **devuelve todas las filas de la tabla de la izquierda (la primera tabla en la cláusula JOIN) y las filas coincidentes de la tabla de la derecha (la segunda tabla en la cláusula JOIN), completando con valores nulos donde no hay coincidencias.**

Funcionamiento del LEFT JOIN:

Al usar LEFT JOIN en una consulta, estás solicitando que se devuelvan todas las filas de la tabla de la izquierda, junto con las filas coincidentes de la tabla de la derecha. Si no hay coincidencias en la tabla de la derecha, los campos relacionados de esa tabla tendrán valores nulos en el resultado.

La sintaxis general del LEFT JOIN es la siguiente:

```
SELECT columnas
```

```
FROM tabla1
```

```
LEFT JOIN tabla2
```

```
ON tabla1.columna = tabla2.columna;
```

La consulta examina cada fila de tabla1 y busca coincidencias en tabla2 basándose en la condición de unión. Si hay una coincidencia, se incluye en el resultado junto con los datos de tabla2. Si no hay coincidencia en tabla2, los campos relacionados de esa tabla tendrán valores nulos en el resultado.

Ejemplo de uso del LEFT JOIN:

Supongamos que tenemos una base de datos de departamentos y empleados, y queremos obtener una lista de todos los empleados y el nombre de su departamento, incluso si algunos empleados no están asignados a un departamento.

```
SELECT E.nombre AS empleado, D.nombre AS departamento
```

```
FROM Empleados E
```

```
LEFT JOIN Departamentos D ON E.id_departamento = D.id_departamento;
```

En este ejemplo, utilizamos un LEFT JOIN para combinar las tablas Empleados y Departamentos. La consulta devuelve todos los empleados junto con el nombre de sus departamentos asignados. Si un empleado no tiene un departamento asignado, su nombre aparecerá en la columna "empleado" y la columna "departamento" tendrá un valor nulo.

Nota: Al igual que con el RIGHT JOIN, el LEFT JOIN es una herramienta poderosa cuando deseas obtener todos los registros de la tabla de la izquierda, incluso si no hay coincidencias en la tabla de la derecha. Esto puede ser especialmente útil para analizar datos en los que uno de los conjuntos puede tener registros faltantes o nulos en la relación con el otro conjunto.

FULL JOIN

El FULL JOIN es otro tipo de operación de unión en SQL que combina registros de dos tablas y devuelve todas las filas de ambas tablas, junto con las filas coincidentes entre ellas. A diferencia del INNER JOIN, que solo devuelve filas con coincidencias, y del LEFT JOIN y RIGHT JOIN, que devuelven todas las filas de una tabla y las coincidencias de la otra, el FULL JOIN **devuelve todas las filas de ambas tablas, llenando con valores nulos donde no hay coincidencias.**

Funcionamiento del FULL JOIN:

El FULL JOIN combina registros de ambas tablas, manteniendo todas las filas de ambas tablas. Si hay coincidencias, se incluyen en el resultado. Si no hay coincidencias en una de las tablas, se rellenan los campos de esa tabla con valores nulos en el resultado.

La sintaxis general del FULL JOIN es la siguiente:

```
SELECT columnas
```

```
FROM tabla1
```

```
FULL JOIN tabla2
```

```
ON tabla1.columna = tabla2.columna;
```

La consulta examina todas las filas de ambas tablas y busca coincidencias basándose en la condición de unión. Si hay una coincidencia, se incluye en el resultado junto con los datos de la otra tabla. Si no hay coincidencia en una de las tablas, los campos relacionados de esa tabla tendrán valores nulos en el resultado.

Ejemplo de uso del FULL JOIN:

Supongamos que tenemos una base de datos con información sobre empleados y proyectos, y queremos obtener una lista de todos los empleados y sus proyectos asignados, incluyendo aquellos empleados que no tienen ningún proyecto asignado.

```
SELECT E.nombre AS empleado, P.nombre AS proyecto  
FROM Empleados E  
FULL JOIN Proyectos P ON E.id_empleado = P.id_empleado;
```

En este ejemplo, estamos utilizando un FULL JOIN para combinar las tablas Empleados y Proyectos. La consulta devuelve todos los empleados junto con los nombres de los proyectos asignados. Si un empleado no tiene proyectos asignados, su nombre aparecerá en la columna "empleado" y la columna "proyecto" tendrá un valor nulo. De manera similar, si un proyecto no tiene un empleado asignado, su nombre aparecerá en la columna "proyecto" y la columna "empleado" tendrá un valor nulo.

Nota: Es importante mencionar que el FULL JOIN no es compatible con todas las bases de datos, y algunas implementaciones pueden requerir alternativas como la combinación de LEFT JOIN y RIGHT JOIN para lograr un resultado similar.

UNION

La cláusula UNION en SQL se utiliza para combinar el resultado de dos o más consultas en una sola tabla de resultados. A diferencia de las operaciones de unión (como INNER JOIN, LEFT JOIN, etc.), que combinan filas de diferentes tablas en función de relaciones específicas, **UNION combina resultados de consultas independientes, eliminando duplicados por defecto.**

Funcionamiento de UNION:

La cláusula UNION permite combinar el resultado de dos o más consultas en una sola tabla resultante. Las consultas deben tener la misma cantidad de columnas y los tipos de datos de las columnas deben ser compatibles entre sí. La consulta resultante incluirá todas las filas de las consultas combinadas, eliminando duplicados por defecto. Si deseas mantener los duplicados, puedes usar UNION ALL.

La sintaxis general del UNION es la siguiente:

```
SELECT columnas
```

```
FROM tabla1
```

```
UNION
```

```
SELECT columnas
```

```
FROM tabla2;
```

Ejemplo de uso de UNION:

Supongamos que tenemos dos tablas diferentes: una que almacena información sobre empleados y otra que almacena información sobre contratistas externos.

Queremos obtener una lista de todas las personas (empleados y contratistas) con sus nombres y correos electrónicos.

```
SELECT nombre, correo
```

```
FROM Empleados
```

```
UNION
```

```
SELECT nombre, correo
```

```
FROM Contratistas;
```

En este ejemplo, estamos utilizando UNION para combinar los resultados de dos consultas independientes. La consulta resultante mostrará una lista de nombres y correos electrónicos de todas las personas, ya sean empleados o contratistas.

Nota: Al usar UNION, es importante que las columnas en las consultas coincidan en términos de cantidad y tipos de datos. Además, recuerda que UNION elimina duplicados por defecto. Si necesitas incluir todos los registros, incluso aquellos que pueden ser duplicados, puedes usar UNION ALL.

Tablas Temporales

Las tablas temporales **son objetos en una base de datos que permiten almacenar datos temporalmente durante el transcurso de una sesión o una tarea específica**. Estas tablas son útiles para almacenar resultados intermedios o temporales de consultas, cálculos, procesamiento o cualquier tarea que requiera una estructura tabular.

Existen dos tipos comunes de tablas temporales: tablas temporales locales y tablas temporales globales. A continuación, te proporcionaré una explicación detallada sobre cada tipo y cómo se utilizan:

Tablas Temporales Locales:

Creación: Las tablas temporales locales se crean dentro de una sesión específica y solo son accesibles por la misma sesión. Se eliminan automáticamente al final de la sesión o cuando se cierra la conexión.

Sintaxis de creación:

```
CREATE TEMPORARY TABLE nombre_tabla (  
    columna1 tipo_dato,  
    columna2 tipo_dato,  
    ...  
);
```

Ejemplo:

```
CREATE TEMPORARY TABLE TempClientes (  
    id INT,  
    nombre VARCHAR(100),  
    email VARCHAR(100)  
);
```

Tablas Temporales Globales:

Creación: Las tablas temporales globales también se crean dentro de una sesión, pero pueden ser accedidas por otras sesiones o conexiones. Permanecen hasta que se eliminen explícitamente o hasta que finalice la sesión que las creó.

Sintaxis de creación:

```
CREATE TEMPORARY TABLE nombre_tabla (  
    columna1 tipo_dato,  
    columna2 tipo_dato,  
    ...  
) GLOBAL;
```

Ejemplo:

```
CREATE TEMPORARY TABLE GlobalTempClientes (  
    id INT,  
    nombre VARCHAR(100),  
    email VARCHAR(100)  
) GLOBAL;
```

Uso de Tablas Temporales:

Las tablas temporales son útiles en situaciones donde necesitas almacenar datos temporales, realizar cálculos intermedios o simplificar tareas complejas. Puedes insertar, actualizar, eliminar y consultar datos en tablas temporales de manera similar a las tablas regulares.

-- Insertar datos en una tabla temporal

```
INSERT INTO TempClientes (id, nombre, email) VALUES (1, 'Juan', 'juan@example.com');
```

-- Consultar datos desde una tabla temporal

```
SELECT * FROM TempClientes;
```

-- Actualizar datos en una tabla temporal

```
UPDATE TempClientes SET email = 'nuevo_email@example.com' WHERE id = 1;
```

-- Eliminar datos de una tabla temporal

```
DELETE FROM TempClientes WHERE id = 1;
```

Consideraciones:

- Las tablas temporales son útiles para tareas temporales, pero no deben confundirse con tablas permanentes que almacenan datos a largo plazo.
- Las características y soporte para tablas temporales pueden variar entre sistemas de gestión de bases de datos (DBMS). Asegúrate de consultar la documentación de tu DBMS para conocer los detalles específicos.
- Ten en cuenta que las tablas temporales locales son más comunes y su alcance está limitado a la sesión que las crea. Las tablas temporales globales son menos comunes y su alcance puede ser más amplio.
- En resumen, las tablas temporales son una herramienta valiosa para almacenar y manipular datos temporales de manera eficiente dentro de una sesión o una tarea específica.

SubConsultas

Subconsultas en SELECT

Las subconsultas en la cláusula SELECT **son una forma de realizar consultas anidadas dentro de otra consulta más grande**. Estas subconsultas permiten obtener resultados de una consulta interna y utilizarlos en la consulta principal para tomar decisiones, filtrar datos, realizar cálculos, entre otros. Las subconsultas pueden ser muy útiles para realizar tareas complejas y obtener información específica de una manera más estructurada.

Funcionamiento de las subconsultas en SELECT:

Las subconsultas en la cláusula SELECT se ejecutan primero, y luego sus resultados se utilizan en la consulta principal. Pueden ir dentro de varias cláusulas, como SELECT, FROM, WHERE, HAVING, entre otras. La subconsulta se trata como una expresión que devuelve un conjunto de resultados.

Ejemplos de uso:

1. Obtener el salario promedio de los empleados y mostrar solo aquellos cuyo salario es superior al promedio:

```
SELECT nombre, salario
```

```
FROM empleados
```

```
WHERE salario > (SELECT AVG(salario) FROM empleados);
```

En este ejemplo, la subconsulta (SELECT AVG(salario) FROM empleados) calcula el salario promedio de todos los empleados. Luego, la consulta principal filtra los

empleados cuyo salario es mayor que el salario promedio calculado en la subconsulta.

2. Obtener detalles de productos junto con la cantidad de ventas de cada producto:

```
SELECT nombre, precio,  
  
       (SELECT COUNT(*) FROM ventas WHERE ventas.id_producto =  
productos.id_producto) AS cantidad_ventas  
  
FROM productos;
```

En este caso, la subconsulta (SELECT COUNT(*) FROM ventas WHERE ventas.id_producto = productos.id_producto) cuenta el número de ventas para cada producto en la tabla ventas. La consulta principal selecciona detalles de productos junto con la cantidad de ventas obtenida de la subconsulta.

3. Obtener empleados cuyo salario está por encima del salario máximo de otro departamento:

```
SELECT nombre, salario  
  
FROM empleados  
  
WHERE salario > (SELECT MAX(salario) FROM empleados WHERE id_departamento =  
2);
```

Aquí, la subconsulta (SELECT MAX(salario) FROM empleados WHERE id_departamento = 2) obtiene el salario máximo en el departamento con id_departamento = 2. La consulta principal selecciona empleados cuyos salarios son mayores que el salario máximo obtenido de la subconsulta.

Las subconsultas en la cláusula SELECT son una herramienta poderosa para combinar y utilizar resultados de diferentes consultas para tomar decisiones y obtener información específica de manera más efectiva.

Subconsultas en FROM

Las subconsultas en la cláusula FROM **se utilizan para crear una tabla derivada o temporal que se utiliza como fuente de datos para la consulta principal.** Estas subconsultas en la cláusula FROM permiten crear una tabla virtual que se puede tratar como cualquier otra tabla en la consulta principal. Esto puede ser útil cuando deseas realizar operaciones más complejas o combinar datos de manera más avanzada.

Funcionamiento de las subconsultas en FROM:

Las subconsultas en la cláusula FROM se ejecutan primero, y el resultado se trata como una tabla temporal. Luego, la consulta principal utiliza esta tabla temporal para realizar operaciones como si fuera una tabla real en la base de datos. Esto permite realizar consultas más avanzadas y complejas.

Ejemplos de uso:

1. Obtener los productos más vendidos junto con la cantidad total de ventas:

```
SELECT P.nombre AS producto, V.cantidad_ventas
```

```
FROM Productos P
```

```
JOIN (SELECT id_producto, SUM(cantidad) AS cantidad_ventas FROM Ventas GROUP BY  
id_producto) V
```

```
ON P.id_producto = V.id_producto
```

ORDER BY V.cantidad_ventas DESC;

En este ejemplo, la subconsulta en la cláusula FROM calcula la suma total de ventas para cada producto en la tabla Ventas. Luego, la consulta principal une esta tabla temporal V con la tabla Productos para obtener los nombres de los productos junto con la cantidad total de ventas. El resultado se ordena por la cantidad de ventas en orden descendente.

2. Obtener empleados y su salario promedio por departamento:

```
SELECT E.id_pleado, E.nombre, E.salario, D.salario_promedio
```

```
FROM Empleados E
```

```
JOIN (SELECT id_departamento, AVG(salario) AS salario_promedio FROM Empleados  
GROUP BY id_departamento) D
```

```
ON E.id_departamento = D.id_departamento;
```

En este caso, la subconsulta en la cláusula FROM calcula el salario promedio por departamento en la tabla Empleados. Luego, la consulta principal une esta tabla temporal D con la tabla Empleados para obtener detalles de empleados junto con el salario promedio por departamento.

3. Obtener la diferencia de ventas entre dos períodos:

```
SELECT V1.fecha, V1.monto - V2.monto AS diferencia_ventas
```

```
FROM (SELECT fecha, SUM(monto) AS monto FROM Ventas WHERE fecha BETWEEN  
'2023-01-01' AND '2023-06-30' GROUP BY fecha) V1
```

```
JOIN (SELECT fecha, SUM(monto) AS monto FROM Ventas WHERE fecha BETWEEN  
'2022-01-01' AND '2022-06-30' GROUP BY fecha) V2
```


ON V1.fecha = V2.fecha;

En este ejemplo, las subconsultas en la cláusula FROM calculan el monto total de ventas para dos períodos diferentes. Luego, la consulta principal une estas dos tablas temporales V1 y V2 basándose en la fecha para obtener la diferencia de ventas entre los dos períodos.

Las subconsultas en la cláusula FROM permiten crear tablas temporales que se utilizan como fuente de datos en la consulta principal, lo que facilita realizar operaciones más avanzadas y realizar análisis más complejos.

Subconsultas en WHERE

Las subconsultas en la cláusula WHERE **son utilizadas para incluir una consulta anidada dentro de otra consulta principal, permitiendo filtrar o condicionar los resultados de acuerdo con los valores obtenidos de la subconsulta**. Estas subconsultas se evalúan para cada fila de la consulta principal y pueden ser útiles para realizar filtrado, comparaciones y toma de decisiones más complejas.

Funcionamiento de las subconsultas en WHERE:

Las subconsultas en la cláusula WHERE se utilizan para comparar valores obtenidos de una subconsulta con los valores de una columna en la consulta principal. La subconsulta se ejecuta para cada fila de la consulta principal y la condición de WHERE se evalúa basándose en los resultados de la subconsulta.

Ejemplos de uso:

1. Obtener los empleados que tienen un salario superior al salario promedio:

```
SELECT id_empleado, nombre, salario  
  
FROM Empleados  
  
WHERE salario > (SELECT AVG(salario) FROM Empleados);
```

En este ejemplo, la subconsulta en la cláusula WHERE calcula el salario promedio de todos los empleados. Luego, la consulta principal selecciona los empleados cuyo salario es mayor que el salario promedio obtenido de la subconsulta.

2. Obtener los productos vendidos más de una vez en un período específico:

```
SELECT id_producto, nombre, COUNT(*) AS cantidad_ventas  
  
FROM Productos  
  
WHERE id_producto IN (SELECT id_producto FROM Ventas WHERE fecha BETWEEN  
'2023-01-01' AND '2023-06-30' GROUP BY id_producto HAVING COUNT(*) > 1)  
  
GROUP BY id_producto, nombre;
```

En este caso, la subconsulta en la cláusula WHERE obtiene los id_producto de la tabla Ventas donde las ventas ocurrieron entre un período específico y donde la cantidad de ventas es mayor que uno. Luego, la consulta principal selecciona los productos cuyos id_producto coinciden con los valores obtenidos de la subconsulta.

3. Obtener los departamentos con más de tres empleados:

```
SELECT id_departamento, nombre  
  
FROM Departamentos  
  
WHERE id_departamento IN (SELECT id_departamento FROM Empleados GROUP BY  
id_departamento HAVING COUNT(*) > 3);
```

En este ejemplo, la subconsulta en la cláusula WHERE obtiene los id_departamento de la tabla Empleados donde la cantidad de empleados es mayor que tres. La consulta principal selecciona los departamentos cuyos id_departamento coinciden con los valores obtenidos de la subconsulta.

Las subconsultas en la cláusula WHERE son una herramienta poderosa para filtrar resultados basándose en información obtenida de otras consultas. Permiten realizar comparaciones y toma de decisiones más avanzadas al filtrar los resultados de la consulta principal.

Bibliografía y Webgrafía

Jacobs, P. (2019). SQL: Guía completa para principiantes de la programación SQL con ejercicios y estudios de casos (Libro En Español/SQL Spanish Book Version) (Spanish Edition). Independently Published.

Moestl Vasilik, S. (2016). Problemas prácticos de SQL: 57 desafíos iniciales, intermedios y avanzados para que los resuelvas utilizando un enfoque de «aprender haciendo». Packt Publishing.

Microsoft. (2023, 20 de enero). Introducción a T-SQL. [Curso en línea]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/modules/introduction-to-transact-sql/>

Microsoft. (2023, 20 de enero). Introducción a las consultas con T-SQL. [Ruta de formación]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/paths/get-started-querying-with-transact-sql/>

Microsoft. (2023, 20 de enero). Introducción a la programación T-SQL. [Curso en línea]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/modules/get-started-transact-sql-programming/>

Glosario

JOIN: Una operación utilizada en bases de datos para combinar filas de dos o más tablas en función de una relación establecida entre ellas. Esto permite obtener datos consolidados y enriquecidos para análisis más completos.

SubConsultas: También conocidas como "subqueries", son consultas anidadas dentro de una consulta principal. Estas subconsultas permiten acceder a datos específicos y realizar operaciones más complejas en pasos discretos.

Tablas Temporales: Son tablas creadas y utilizadas temporalmente para almacenar datos intermedios o realizar cálculos en el proceso de análisis. Estas tablas son útiles para manejar datos que cambian constantemente sin afectar la integridad de la base de datos principal.

UNION: Una operación que permite combinar los resultados de dos o más consultas independientes para obtener una vista integral de la información. Esto facilita la obtención de datos consolidados y más completos.